```
bc4278_nyu_edu@nyu-dataproc-m:~/6_HW$ spark-shell --master yarn --deploy-mode client --package
s   org.apache.poi:poi:5.2.5 < Ingest.scala
:: loading settings :: url = jar:file:/usr/lib/spark/jars/ivy-2.5.2.jar!/org/apache/ivy/core/s
ettings/ivysettings.xml
Ivy Default Cache set to: /home/bc4278_nyu_edu/.ivy2/cache
The jars for the packages stored in: /home/bc4278_nyu_edu/.ivy2/jars
org.apache.poi#poi added as a dependency
:: resolving dependencies :: org.apache.spark#spark-submit-parent-14a894f6-0c98-40aa-b59e-a915
62d19096;1.0
        confs: [default]
        found org.apache.poi#poi;5.2.5 in central
        found commons-codec#commons-codec;1.16.0 in central
        found org.apache.commons#commons-collections4;4.4 in central
        found org.apache.commons#commons-math3;3.6.1 in central
        found commons-io#commons-io;2.15.0 in central
        found com.zaxxer#SparseBitSet;1.3 in central
        found org.apache.logging.log4j#log4j-api;2.21.1 in central
:: resolution report :: resolve 444ms :: artifacts dl 71ms
        :: modules in use:
        com.zaxxer#SparseBitSet;1.3 from central in [default]
        commons-codec#commons-codec;1.16.0 from central in [default]
        commons-io#commons-io;2.15.0 from central in [default]
        org.apache.commons#commons-collections4;4.4 from central in [default]
        org.apache.commons#commons-math3;3.6.1 from central in [default]
        org.apache.logging.log4j#log4j-api;2.21.1 from central in [default]
        org.apache.poi#poi;5.2.5 from central in [default]
        ---------------------------------------------------------------------
        |                  |            modules            ||   artifacts   |
        |       conf       | number| search|dwnlded|evicted|| number|dwnlded|
        ---------------------------------------------------------------------
        |      default     |   7   |   0   |   0   |   0   ||   7   |   0   |
        ---------------------------------------------------------------------
:: retrieving :: org.apache.spark#spark-submit-parent-14a894f6-0c98-40aa-b59e-a91562d19096
        confs: [default]
        0 artifacts copied, 7 already retrieved (0kB/23ms)
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/05 05:11:48 INFO SparkEnv: Registering MapOutputTracker
26/04/05 05:11:48 INFO SparkEnv: Registering BlockManagerMaster
26/04/05 05:11:48 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/04/05 05:11:49 INFO SparkEnv: Registering OutputCommitCoordinator
```

```
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.3
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 11.0.20.1)
Type in expressions to have them evaluated.
Type :help for more information.

scala> import org.apache.poi.hssf.usermodel.HSSFWorkbook
import org.apache.poi.hssf.usermodel.HSSFWorkbook

scala> import org.apache.spark.sql.{Row, SparkSession}
import org.apache.spark.sql.{Row, SparkSession}

scala> import org.apache.spark.sql.types._
import org.apache.spark.sql.types._

scala> import scala.collection.mutable.ArrayBuffer
import scala.collection.mutable.ArrayBuffer

scala>

scala> val hdfsBase = "6_HW"
hdfsBase: String = 6_HW

scala> val outBase = s"${hdfsBase}/ingested"
outBase: String = 6_HW/ingested

scala> val conf = spark.sparkContext.hadoopConfiguration
conf: org.apache.hadoop.conf.Configuration = Configuration: core-default.xml, core-site.xml, m
apred-default.xml, mapred-site.xml, yarn-default.xml, yarn-site.xml, hdfs-default.xml, hdfs-rb
f-default.xml, hdfs-site.xml, hdfs-rbf-site.xml, resource-types.xml, __spark_hadoop_conf__.xml

scala> val fs = org.apache.hadoop.fs.FileSystem.get(conf)
fs: org.apache.hadoop.fs.FileSystem = DFS[DFSClient[clientName=DFSClient_NONMAPREDUCE_-6726200
77_1, ugi=bc4278_nyu_edu (auth:SIMPLE)]]

scala>
```

```
scala> val eptmcFiles = Map(
     |   2019 -> s"${hdfsBase}/EPTMC/eptmc2019tablas.xls",
     |   2020 -> s"${hdfsBase}/EPTMC/eptmc2020.tablas.xls",
     |   2021 -> s"${hdfsBase}/EPTMC/eptmc2021.tablas.xls",
     |   2022 -> s"${hdfsBase}/EPTMC/eptmc2022.tablas.xls",
     |   2023 -> s"${hdfsBase}/EPTMC/eptmc2023.tablas.xls",
     |   2024 -> s"${hdfsBase}/EPTMC/eptmc2024.tablas.xls"
     | )
eptmcFiles: scala.collection.immutable.Map[Int,String] = Map(2020 -> 6_HW/EPTMC/eptmc2020.tabl
as.xls, 2024 -> 6_HW/EPTMC/eptmc2024.tablas.xls, 2023 -> 6_HW/EPTMC/eptmc2023.tablas.xls, 2022
 -> 6_HW/EPTMC/eptmc2022.tablas.xls, 2019 -> 6_HW/EPTMC/eptmc2019tablas.xls, 2021 -> 6_HW/EPTM
C/eptmc2021.tablas.xls)

scala>

scala> def openWb(path: String): HSSFWorkbook = {
     |   val is = fs.open(new org.apache.hadoop.fs.Path(path))
     |   val wb = new HSSFWorkbook(is); is.close(); wb
     | }
openWb: (path: String)org.apache.poi.hssf.usermodel.HSSFWorkbook

scala>

scala> def cleanCell(cell: org.apache.poi.ss.usermodel.Cell): Option[Double] = {
     |   if (cell == null) return None
     |   cell.getCellType match {
     |     case org.apache.poi.ss.usermodel.CellType.NUMERIC => Some(cell.getNumericCellValue)

     |     case org.apache.poi.ss.usermodel.CellType.STRING =>
     |       val s = cell.getStringCellValue.trim
     |       if (s == "" || s == ".." || s == "." || s == "-") None
     |       else scala.util.Try(s.toDouble).toOption
     |     case _ => None
     |   }
     | }
cleanCell: (cell: org.apache.poi.ss.usermodel.Cell)Option[Double]

scala>
```

```
scala> val regionNorm = Map(
     |   "Andaluc" -> "Andalucia", "Arag" -> "Aragon", "Asturias" -> "Asturias",
     |   "Balears" -> "Baleares", "Canarias" -> "Canarias", "Cantabria" -> "Cantabria",
     |   "Castilla-La Mancha" -> "Castilla-La Mancha", "Castilla y Le" -> "Castilla y Leon",
     |   "Catalu" -> "Cataluna", "Comunidad Valenciana" -> "Com. Valenciana",
     |   "Extremadura" -> "Extremadura", "Galicia" -> "Galicia",
     |   "Madrid" -> "Madrid", "Murcia" -> "Murcia", "Navarra" -> "Navarra",
     |   "Pa" -> "Pais Vasco", "Rioja" -> "Rioja", "Ceuta" -> "Ceuta y Melilla"
     | )
regionNorm: scala.collection.immutable.Map[String,String] = Map(Andaluc -> Andalucia, Balears
-> Baleares, Extremadura -> Extremadura, Arag -> Aragon, Comunidad Valenciana -> Com. Valencia
na, Ceuta -> Ceuta y Melilla, Pa -> Pais Vasco, Catalu -> Cataluna, Castilla y Le -> Castilla
y Leon, Galicia -> Galicia, Navarra -> Navarra, Murcia -> Murcia, Cantabria -> Cantabria, Cana
rias -> Canarias, Rioja -> Rioja, Castilla-La Mancha -> Castilla-La Mancha, Asturias -> Asturi
as, Madrid -> Madrid)

scala> def normRegion(raw: String): String = {
     |   val s = raw.trim
     |   if (s == "TOTAL") return null
     |   for ((prefix, canon) <- regionNorm) { if (s.startsWith(prefix)) return canon }
     |   s
     | }
normRegion: (raw: String)String

scala>
```

```
scala> // ============================================================

scala> // 1. DISTANCE BANDS (from eptmc2024 only)

scala> // ============================================================

scala> println(">> Ingesting DISTANCE BANDS...")
>> Ingesting DISTANCE BANDS...

scala> {
     |   val dbMetrics = Map("operations_total" -> "4.9", "operations_loaded" -> "4.10",
     |     "tonnes_thousands" -> "4.11", "tonne_km_millions" -> "4.12")
     |   val bands = Array("Total", "Menos de 50 km", "De 50 a 149 km", "De 150 a 500 km", "Ma
s de 500 km")
     |   val bandCols = Array(1, 2, 3, 4, 5)
     |   val sections = Map("total" -> (15, 22), "cuenta_ajena" -> (25, 32), "cuenta_propia" -
> (35, 42))
     |   val wb = openWb(eptmcFiles(2024))
     |   val rows = new ArrayBuffer[(Int, String, String, String, Double)]()
     |   for ((metric, sheetName) <- dbMetrics) {
     |     val sh = wb.getSheet(sheetName)
     |     for ((svcType, (rs, re)) <- sections; r <- rs until re) {
     |       val row = sh.getRow(r)
     |       if (row != null) {
     |         val c0 = row.getCell(0)
     |         if (c0 != null && c0.getCellType == org.apache.poi.ss.usermodel.CellType.NUMERI
C) {
     |           val year = c0.getNumericCellValue.toInt
     |           if (year >= 2018 && year <= 2024)
     |             for ((band, col) <- bands.zip(bandCols))
     |               cleanCell(row.getCell(col)).foreach(v => rows += ((year, band, svcType, m
etric, v)))
     |         }
     |       }
     |     }
     |   }
     |   wb.close()
     |   val schema = StructType(Array(StructField("year", IntegerType), StructField("distance
_band", StringType),
     |     StructField("service_type", StringType), StructField("metric", StringType), StructF
ield("value", DoubleType)))
```

```
     |     StructField("service_type", StringType), StructField("metric", StringType), StructF
ield("value", DoubleType)))
     |   val df = spark.createDataFrame(sc.parallelize(rows.map(r => Row(r._1, r._2, r._3, r._
4, r._5))), schema)
     |   df.coalesce(1).write.mode("overwrite").option("header", "true").csv(s"${outBase}/dist
ance_bands")
     |   println(s"   DISTANCE BANDS: ${df.count()} rows")
     | }
   DISTANCE BANDS: 420 rows

scala>

scala> // ============================================================

scala> // 2. PRICE INDEX (from eptmc2020 + eptmc2024)

scala> // ============================================================

scala> println(">> Ingesting PRICE INDEX...")
>> Ingesting PRICE INDEX...

scala> {
     |   val bands = Array("Total", "50 km y menos", "Entre 51 y 100 km", "Entre 101 y 200 km"
, "Entre 201 y 300 km", "Mas de 300 km")
     |   val qMap = Map("1T" -> "Q1", "2T" -> "Q2", "3T" -> "Q3", "4T" -> "Q4")
     |   def extractQ(filePath: String, yMin: Int, yMax: Int): ArrayBuffer[(Int, String, Strin
g, Double)] = {
     |     val wb = openWb(filePath); val sh = wb.getSheet("2.9")
     |     val rows = new ArrayBuffer[(Int, String, String, Double)]()
     |     var curYear: Option[Int] = None
     |     for (r <- 23 until sh.getLastRowNum + 1) {
     |       val row = sh.getRow(r)
     |       if (row != null) {
     |         val c0 = row.getCell(0); val c1 = row.getCell(1)
     |         if (c0 != null && c0.getCellType == org.apache.poi.ss.usermodel.CellType.NUMERI
C)
     |           curYear = Some(c0.getNumericCellValue.toInt)
     |         else if (c0 != null && c0.getCellType == org.apache.poi.ss.usermodel.CellType.S
TRING) {
     |           val s = c0.getStringCellValue.trim
     |           if (s.nonEmpty && s.forall(_.isDigit)) curYear = Some(s.toInt)
     |         }
```

```
     |           curYear = Some(c0.getNumericCellValue.toInt)
     |         else if (c0 != null && c0.getCellType == org.apache.poi.ss.usermodel.CellType.S
TRING) {
     |           val s = c0.getStringCellValue.trim
     |           if (s.nonEmpty && s.forall(_.isDigit)) curYear = Some(s.toInt)
     |         }
     |         if (c1 != null && c1.getCellType == org.apache.poi.ss.usermodel.CellType.STRING
)
     |           qMap.get(c1.getStringCellValue.trim).foreach { q =>
     |             curYear.filter(y => y >= yMin && y <= yMax).foreach { year =>
     |               for (i <- 0 until 6) cleanCell(row.getCell(2 + i)).foreach(v => rows += (
(year, q, bands(i), v)))
     |             }
     |           }
     |       }
     |     }
     |     wb.close(); rows
     |   }
     |   val allRows = extractQ(eptmcFiles(2020), 2019, 2020) ++ extractQ(eptmcFiles(2024), 20
21, 2024)
     |   val schema = StructType(Array(StructField("year", IntegerType), StructField("quarter"
, StringType),
     |     StructField("distance_band", StringType), StructField("price_index", DoubleType)))
     |   val df = spark.createDataFrame(sc.parallelize(allRows.map(r => Row(r._1, r._2, r._3,
r._4))), schema)
     |   df.coalesce(1).write.mode("overwrite").option("header", "true").csv(s"${outBase}/pric
e_index")
     |   println(s"   PRICE INDEX: ${df.count()} rows")
     | }
   PRICE INDEX: 144 rows

scala>
```

```
scala> // ============================================================

scala> // 3. REGIONAL (sheets 6.1-6.4, all 6 files)

scala> // ============================================================

scala> println(">> Ingesting REGIONAL...")
>> Ingesting REGIONAL...

scala> {
     |   val rMetrics = Map("operations_total" -> "6.1", "operations_empty" -> "6.2",
     |     "tonnes_thousands" -> "6.3", "tonne_km_millions" -> "6.4")
     |   val dispCols = Map("intrarregional_total" -> 1, "intramunicipal" -> 2,
     |     "intermunicipal_intrarregional" -> 3, "interregional_recibido" -> 5,
     |     "interregional_expedido" -> 6, "internacional_recibido" -> 8, "internacional_expedi
do" -> 9)
     |   val rows = new ArrayBuffer[(Int, String, String, String, Double)]()
     |   for ((year, fp) <- eptmcFiles) {
     |     val wb = openWb(fp)
     |     for ((metric, sn) <- rMetrics) {
     |       val sh = wb.getSheet(sn)
     |       for (r <- 17 until math.min(36, sh.getLastRowNum + 1)) {
     |         val row = sh.getRow(r)
     |         if (row != null) {
     |           val c0 = row.getCell(0)
     |           if (c0 != null && c0.getCellType == org.apache.poi.ss.usermodel.CellType.STRI
NG) {
     |             val region = normRegion(c0.getStringCellValue)
     |             if (region != null)
     |               for ((dt, col) <- dispCols) cleanCell(row.getCell(col)).foreach(v => rows
 += ((year, region, dt, metric, v)))
     |           }
     |         }
     |       }
     |     }
     |     wb.close()
     |   }
     |   val schema = StructType(Array(StructField("year", IntegerType), StructField("region",
 StringType),
```

```
     |   val schema = StructType(Array(StructField("year", IntegerType), StructField("region",
 StringType),
     |     StructField("displacement_type", StringType), StructField("metric", StringType), St
ructField("value", DoubleType)))
     |   val df = spark.createDataFrame(sc.parallelize(rows.map(r => Row(r._1, r._2, r._3, r._
4, r._5))), schema)
     |   df.coalesce(1).write.mode("overwrite").option("header", "true").csv(s"${outBase}/regi
onal")
     |   println(s"   REGIONAL: ${df.count()} rows")
     | }
   REGIONAL: 2844 rows

scala>

scala> // ============================================================

scala> // 4. CORRIDORS (sheets 6.5-6.8 + s supplements, all 6 files)

scala> // ============================================================

scala> println(">> Ingesting CORRIDORS...")
>> Ingesting CORRIDORS...

scala> {
     |   val cMetrics = Map("transport_operations_total" -> ("6.5", "6.5s"),
     |     "transport_operations_empty" -> ("6.6", "6.6s"),
     |     "tonnes_thousands" -> ("6.7", "6.7s"), "tonne_km_millions" -> ("6.8", "6.8s"))
     |   val regions = Array("Andalucia", "Aragon", "Asturias", "Baleares", "Canarias",
     |     "Cantabria", "Castilla-La Mancha", "Castilla y Leon", "Cataluna",
     |     "Com. Valenciana", "Extremadura", "Galicia", "Madrid",
     |     "Murcia", "Navarra", "Pais Vasco", "Rioja", "Ceuta y Melilla")
     |   val excluded = Set("Baleares", "Canarias", "Ceuta y Melilla")
     |   val leftCols = (1 to 10).toArray; val rightCols = (0 to 7).toArray
     |
     |   def extractMatrix(wb: HSSFWorkbook, shL: String, shR: String): Map[(Int,Int), Double]
 = {
     |     val sL = wb.getSheet(shL); val sR = wb.getSheet(shR)
     |     val m = scala.collection.mutable.Map[(Int,Int), Double]()
     |     for (ri <- 0 until 18) {
     |       val r = 17 + ri
     |       val rL = sL.getRow(r); val rR = sR.getRow(r)
```

```
     |       val rL = sL.getRow(r); val rR = sR.getRow(r)
     |       if (rL != null) for ((c, ci) <- leftCols.zipWithIndex) cleanCell(rL.getCell(c)).f
oreach(v => m((ri, ci)) = v)
     |       if (rR != null) for ((c, co) <- rightCols.zipWithIndex) cleanCell(rR.getCell(c)).
foreach(v => m((ri, 10+co)) = v)
     |     }
     |     m.toMap
     |   }
     |
     |   val rows = new ArrayBuffer[(Int, String, String, String, Double)]()
     |   for ((year, fp) <- eptmcFiles) {
     |     val wb = openWb(fp)
     |     for ((metric, (sl, sr)) <- cMetrics) {
     |       val matrix = extractMatrix(wb, sl, sr)
     |       for (oi <- regions.indices; di <- regions.indices if oi != di) {
     |         val o = regions(oi); val d = regions(di)
     |         if (!excluded(o) && !excluded(d)) matrix.get((oi, di)).foreach(v => rows += ((y
ear, o, d, metric, v)))
     |       }
     |     }
     |     wb.close()
     |   }
     |   val schema = StructType(Array(StructField("year", IntegerType), StructField("origin_r
egion", StringType),
     |     StructField("destination_region", StringType), StructField("metric", StringType), S
tructField("value", DoubleType)))
     |   val df = spark.createDataFrame(sc.parallelize(rows.map(r => Row(r._1, r._2, r._3, r._
4, r._5))), schema)
     |   df.coalesce(1).write.mode("overwrite").option("header", "true").csv(s"${outBase}/corr
idors")
     |   println(s"   CORRIDORS: ${df.count()} rows")
     | }
   CORRIDORS: 4863 rows

scala>
```

```
scala> // ============================================================

scala> // 5. CARGO (sheets 7.4p1-p8, 7.5p1-p8, all 6 files)

scala> // ============================================================

scala> println(">> Ingesting CARGO...")
>> Ingesting CARGO...

scala> {
     |   val cargoL = Array("Agricolas y animales vivos", "Alimenticios y forrajes",
     |     "Combustibles minerales solidos", "Productos petroliferos", "Minerales y residuos p
ara refundicion")
     |   val cargoR = Array("Productos metalurgicos", "Minerales y materiales para construccio
n",
     |     "Abonos", "Productos quimicos", "Maquinas, vehiculos, objetos manufacturados")
     |   val lCols = Array(2,3,4,5,6); val rCols = Array(0,1,2,3,4)
     |
     |   def findBlocks(sh: org.apache.poi.ss.usermodel.Sheet): ArrayBuffer[(Int, String)] = {

     |     val blocks = new ArrayBuffer[(Int, String)]()
     |     for (r <- 16 until sh.getLastRowNum) {
     |       val row = sh.getRow(r)
     |       if (row != null) { val c = row.getCell(0)
     |         if (c != null && c.getCellType == org.apache.poi.ss.usermodel.CellType.STRING)
{
     |           val lbl = c.getStringCellValue.trim
     |           if (lbl.nonEmpty && !lbl.startsWith("(") && !lbl.startsWith("Fuente")) {
     |             val nr = sh.getRow(r+1)
     |             if (nr != null) { val nc = nr.getCell(0)
     |               if (nc != null && nc.getCellType == org.apache.poi.ss.usermodel.CellType.
STRING &&
     |                   nc.getStringCellValue.contains("Total transportado"))
     |                 blocks += ((r, normRegion(lbl)))
     |             }
     |           }
     |         }
     |       }
     |     }
     |     blocks
     |   }
```

```
     |       }
     |       blocks
     |     }
     |
     |     def extractTable(wb: HSSFWorkbook, pfx: String, year: Int, metric: String): ArrayBuff
er[(Int,String,String,String,String,Double)] = {
     |       val rows = new ArrayBuffer[(Int,String,String,String,String,Double)]()
     |       for (i <- 1 to 7 by 2) {
     |         val shO = wb.getSheet(s"${pfx}p${i}"); val shE = wb.getSheet(s"${pfx}p${i+1}")
     |         for ((hr, region) <- findBlocks(shO)) {
     |           for ((dir, dr) <- Array(("expedido", hr+7), ("recibido", hr+8))) {
     |             if (dr <= shO.getLastRowNum) {
     |               val rO = shO.getRow(dr); val rE = shE.getRow(dr)
     |               if (rO != null) for ((c, idx) <- lCols.zipWithIndex) cleanCell(rO.getCell(c
)).foreach(v => rows += ((year, region, dir, cargoL(idx), metric, v)))
     |               if (rE != null) for ((c, idx) <- rCols.zipWithIndex) cleanCell(rE.getCell(c
)).foreach(v => rows += ((year, region, dir, cargoR(idx), metric, v)))
     |             }
     |           }
     |         }
     |       }
     |       rows
     |     }
     |
     |     val rows = new ArrayBuffer[(Int,String,String,String,String,Double)]()
     |     for ((year, fp) <- eptmcFiles) {
     |       val wb = openWb(fp)
     |       rows ++= extractTable(wb, "7.4", year, "operations")
     |       rows ++= extractTable(wb, "7.5", year, "tonnes_thousands")
     |       wb.close()
     |     }
     |     val schema = StructType(Array(StructField("year", IntegerType), StructField("region",
 StringType),
     |       StructField("direction", StringType), StructField("cargo_class", StringType),
     |       StructField("metric", StringType), StructField("value", DoubleType)))
     |     val df = spark.createDataFrame(sc.parallelize(rows.map(r => Row(r._1, r._2, r._3, r._
4, r._5, r._6))), schema)
     |     df.coalesce(1).write.mode("overwrite").option("header", "true").csv(s"${outBase}/carg
o")
     |     println(s"   CARGO: ${df.count()} rows")
     | }
   CARGO: 3478 rows
```

```
scala> // ============================================================

scala> // 6. ACOTRAM (.obs files)

scala> // ============================================================

scala> println(">> Ingesting ACOTRAM...")
>> Ingesting ACOTRAM...

scala> {
     |   val directF = Map("vehicle_type"->129, "km_anual"->135, "pct_carga"->136,
     |     "horas_anuales"->137, "personal_conduccion_eur"->177, "dietas_conductor_eur"->182,
     |     "seguros_eur"->192, "costes_fiscales_eur"->202, "fuel_price_per_litre"->206,
     |     "fuel_consumption_coeff"->207, "peajes_eur"->229, "urea_eur"->235,
     |     "costes_indirectos_eur"->240, "period"->256)
     |   val formulaF = Map("vehicle_price_eur"->142, "semitrailer_price_eur"->159, "interest_
rate_pct"->148)
     |
     |   def parseVal(raw: String): Any = {
     |     val s = raw.trim
     |     if (s.startsWith("\"") && s.endsWith("\"")) return s.substring(1, s.length-1)
     |     if (s.startsWith("#") && s.endsWith("#")) return s.substring(1, s.length-1)
     |     try { if (s.contains(".")) s.toDouble else s.toInt } catch { case _: Exception => s
 }
     |   }
     |   def toD(v: Any): Double = v match { case d:Double=>d; case i:Int=>i.toDouble; case s:
String=>s.toDouble; case _=>0.0 }
     |
     |   val basePath = new org.apache.hadoop.fs.Path(s"${hdfsBase}/AcotramData")
     |   val folders = fs.listStatus(basePath).filter(_.isDirectory).map(_.getPath).sortBy(_.g
etName)
     |   val allRecs = new ArrayBuffer[Map[String,Any]]()
     |
     |   for (folder <- folders) {
     |     val fp = new org.apache.hadoop.fs.Path(folder, "Articulado Carga General.obs")
     |     if (fs.exists(fp)) {
     |       val is = fs.open(fp)
     |       val rd = new java.io.BufferedReader(new java.io.InputStreamReader(is, "ISO-8859-1
"))
     |       val lines = new ArrayBuffer[String]()
```

```
     |       val lines = new ArrayBuffer[String]()
     |       var ln = rd.readLine(); while (ln != null) { lines += ln; ln = rd.readLine() }; r
d.close()
     |       if (lines.length >= 256) {
     |         val rec = scala.collection.mutable.Map[String,Any]()
     |         for ((f, n) <- directF) rec(f) = parseVal(lines(n-1))
     |         for ((f, n) <- formulaF) rec(f) = parseVal(lines(n-1))
     |         rec("km_en_carga") = toD(rec("km_anual")) * toD(rec("pct_carga")) / 100.0
     |         rec("combustible_eur") = toD(rec("fuel_price_per_litre")) * toD(rec("fuel_consu
mption_coeff")) * toD(rec("km_anual"))
     |         allRecs += rec.toMap
     |       }
     |     }
     |   }
     |
     |   val outFields = Array("period","vehicle_type","km_anual","pct_carga","km_en_carga",
     |     "horas_anuales","personal_conduccion_eur","dietas_conductor_eur","seguros_eur",
     |     "costes_fiscales_eur","peajes_eur","urea_eur","costes_indirectos_eur",
     |     "vehicle_price_eur","semitrailer_price_eur","fuel_price_per_litre",
     |     "fuel_consumption_coeff","interest_rate_pct","combustible_eur")
     |   val schema = StructType(outFields.map(f =>
     |     if (f=="period"||f=="vehicle_type") StructField(f, StringType) else StructField(f,
DoubleType)))
     |   val sparkRows = allRecs.sortBy(_("period").toString).map { rec =>
     |     Row.fromSeq(outFields.map(f => if (f=="period"||f=="vehicle_type") rec(f).toString
else toD(rec(f))))
     |   }
     |   val df = spark.createDataFrame(sc.parallelize(sparkRows), schema)
     |   df.coalesce(1).write.mode("overwrite").option("header", "true").csv(s"${outBase}/acot
ram")
     |   println(s"   ACOTRAM: ${df.count()} rows")
     | }
   ACOTRAM: 31 rows

scala>

scala> println("\n== ALL 6 DATASETS INGESTED ==")
```